BraidTeX — User Manual

Braid and little-disc diagrams as ordinary mathematics

version 0.2.0

Contents

1 What BraidTeX does **2**

2 Installation and basic setup **2**

2.1 Requirements . . . 2

2.2 Install . . . 2

2.3 A first document . . . 2

3 Drawing individual braids **3**

3.1 Crossings and the convention . . . 3

3.2 Identity, unlabeled and inferred braids . . . 4

3.3 Output configurations are checked . . . 4

3.4 Colours . . . 5

3.5 Parentheses decide the layout . . . 5

3.6 Axes, marked points, annotations, crossing labels . . . 6

3.7 Inline use . . . 6

4 Composition: an inner braid replaces a strand **6**

4.1 Where the inner braid sits, how far the marking runs, appearance . . . 7

5 Standard identities **8**

5.1 Braid relations and basic crossing identities . . . 8

5.2 Associativity, pentagon, hexagon . . . 9

6 Little 2-disc diagrams **10**

6.1 Operadic composition . . . 11

7 Customizing **11**

7.1 Examples . . . 12

8 Gallery **12**

9 Troubleshooting **14**

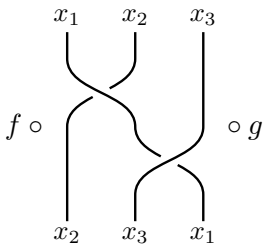
10 Reference **15**

10.1 Command line 15
10.2 Python 15
10.3 Limitations 16

1 What BraidTeX does

BraidTeX turns a few lines of LaTeX into a braid diagram inside your formula:

```
f \circ \braid{ \bin{x_1 x_2 x_3} \bcr{1} \bcr{-2} \bout{x_2 x_3 x_1} } \circ g
```



Strand connectivity, spacing (from parentheses), crossings with clean vector gaps, colours, labels, caching and the TikZ output are generated for you. Everything is *vector* graphics.

How it works — read this once. TeX cannot run programs, and BraidTeX does not use shell escape. `\braid{...}` is therefore a *preprocessing* construct: the command `braidtex compile main.tex` scans your source, validates every braid, draws each one into a small TikZ file under `braids/`, writes a transformed *copy* of your source in which each `\braid{...}` is replaced by `\braidfig{braids/bt-<hash>}`, and runs LaTeX on the copy. Your original files are never modified. Running plain `pdflatex` on an unprocessed file prints a warning and a placeholder box instead of the diagram.

2 Installation and basic setup

2.1 Requirements

Python $\geq$ 3.9 (no third-party packages), a TeX distribution with `tikz`, `xcolor` and `standalone`; `latexmk` is used if present. Engines: pdfLaTeX, LuaLaTeX, XeLaTeX.

2.2 Install

```
pip install . # from the BraidTeX directory (pip install -e .[test] to develop)
braidtex --version
```

2.3 A first document

Save this as `main.tex` in any directory (it may live outside the BraidTeX directory; paths with spaces are fine):

```
\documentclass{article}
\usepackage{amsmath}
```

```
\usepackage{braidtex} % the only thing to add to the preamble
\begin{document}
\[ f \circ \braid{\bin{x_1 x_2 x_3}\bcr{1}\bcr{-2}\bout{x_2 x_3 x_1}} \circ g \]
\end{document}
```

```
braidtex compile main.tex # -> main.pdf (assets in braids/, aux files in .braidtex-
    build/)
braidtex compile main.tex -v # show which diagrams were generated / reused
braidtex transform main.tex # only preprocess; the result is in .braidtex-build/main.
    tex
braidtex clean # delete braids/ and .braidtex-build/
```

Files. The engine runs with your project directory as working directory, so relative `\input`, `\includegraphics` and bibliography paths keep working. Files included with `\input` or `\include` are transformed too. Unchanged diagrams are *reused* from `braids/` on the next run (the cache key covers every property that affects the drawing); change one braid and only that one is redrawn. To run `pdflatex` yourself, copy the style file once: `cp "$(braidtex sty)/braidtex.sty" .`

3 Drawing individual braids

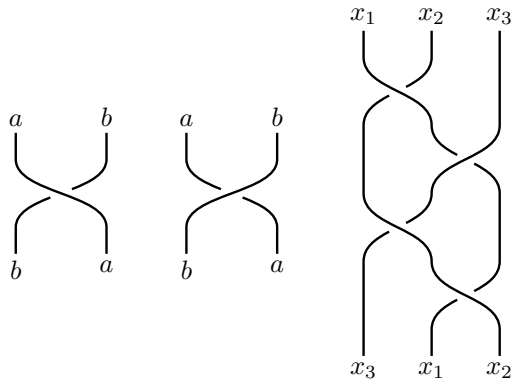
All commands live inside `\braid{...}`; they are short and independent. Commands:

`\bin{cfg}`	input configuration (labels and parentheses); infers the number of strands
`\bout{cfg}`	output configuration, *validated* against the braid; optional
`\bn{N}`	strand count for unlabeled braids
`\bcr{1 -2 1}`	crossings, top to bottom; $i = \sigma_i$, $-i = \sigma_i^{-1}$ (may be repeated)
`\bcol{ids}{color}`	colour of the given strands; `\bcol{*}{c}` sets the main colour
`\bnote{k}{tex}`	annotation attached to strand k
`\blab{in|out}{labels}`	display-only labels (never checked); `*` hides one
`\bax{frame}` / `\bax{h}`	reference axes
`\bpt{in|out}{k}{label}`	marked point at endpoint k
`\bsub{k}{inner}`	replace strand k by an inner braid (Section 4)
`\bset{key=value,...}`	local options (Section 7)
`\braidsetup{key=value,...}`	defaults for all following braids
`\braidid{name}{options}`	standard identities (Section 5)

3.1 Crossings and the convention

Words are read **top to bottom**. At a positive σ_i the strand entering from the *left* passes **over**; σ_i^{-1} reverses this. Strand k is the k-th input endpoint and keeps its identity (and colour) through all crossings.

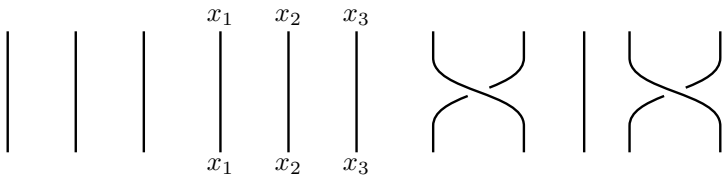
```
\braid{\bin{a b}\bcr{1}} \qquad \braid{\bin{a b}\bcr{-1}} \qquad
\braid{\bin{x_1 x_2 x_3}\bcr{1 -2 1 2}}
```



3.2 Identity, unlabeled and inferred braids

The number of strands is inferred from `\bin`, `\bout` or the crossings. *Omitted*, *empty* and *unlabeled* are different: `\bn{3}` (or `* * *`) is three unlabeled strands; `\bin{}` is the empty configuration (zero strands); an omitted `\bin` is inferred from the rest (crossings only: $\max|i|+1$).

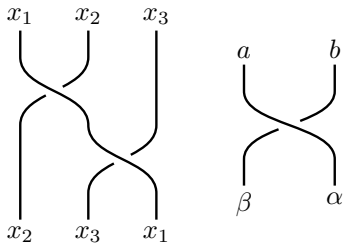
```
\braid{\bn{3}} \qquad \braid{\bin{x_1 x_2 x_3}} \qquad \braid{\bcr{1}} \qquad \braid{\
    bin{(* *) *}\bcr{2}}
```



3.3 Output configurations are checked

Labels follow the strands. `\bout` states the expected output and is *validated*: a wrong output is an error, never a misleading picture. To change only what is *shown* use `\blab`.

```
\braid{\bin{x_1 x_2 x_3}\bcr{1}\bcr{2}\bout{x_2 x_3 x_1}} \qquad
\braid{\bin{x_1 x_2}\bcr{1}\blab{in}{a b}\blab{out}{\beta \alpha}}
```

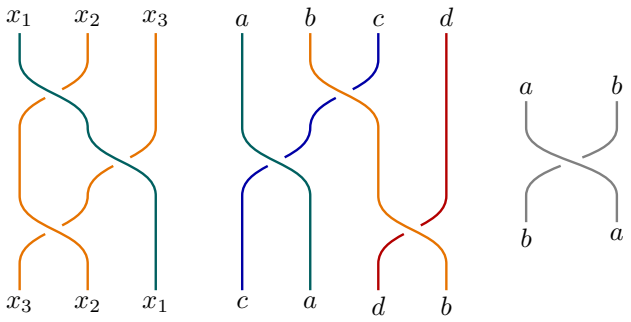


A mismatch such as `\bout{x_1 x_2 x_3}` after `\bcr{1}` stops the build with `main.tex:12:5: output endpoint 1 is labeled 'x_1' but the strand arriving there ... is 'x_2'`.

3.4 Colours

Colours are TikZ/xcolor expressions such as `teal!75!black`. They follow strands through crossings. `\bset{palette}` applies a default colour scheme; `\bcol{*}{c}` sets the main colour.

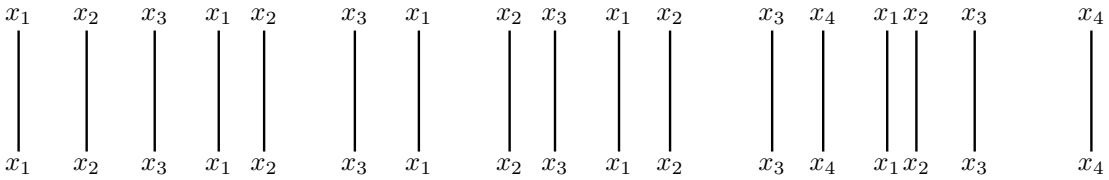
```
\braid{\bin{x_1 x_2 x_3}\bcr{1 2 1}\bcol{1}{teal!75!black}\bcol{2 3}{orange!90!black}}
    \qquad
\braid{\bin{a b c d}\bset{palette}\bcr{2 1 3}} \qquad
\braid{\bin{a b}\bcr{1}\bcol{*}{gray}}
```



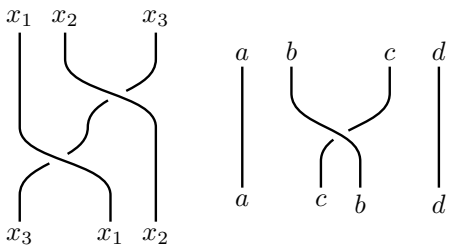
3.5 Parentheses decide the layout

Whitespace separates atoms; parentheses group. No layout option is needed: gap between neighbouring endpoints is $\max(\rho^d, w_{\min})$ with d the depth of their lowest common ancestor (flat configurations are uniform; nesting gives tighter clusters). Input and output are laid out *independently*.

```
\braid{\bin{x_1 x_2 x_3}} \quad \braid{\bin{(x_1 x_2) x_3}} \quad \braid{\bin{x_1 (x_2
    x_3)}} \quad
\braid{\bin{(x_1 x_2)(x_3 x_4)}} \quad \braid{\bin{((x_1 x_2) x_3) x_4}}
```




```
\braid{\bin{(x_1 x_2) x_3}\bcr{2 1}\bout{x_3 (x_1 x_2)}} \qquad
\braid{\bin{(a b)(c d)}\bcr{2}\bout{a (c b) d}\bset{width=3.6cm}}
```

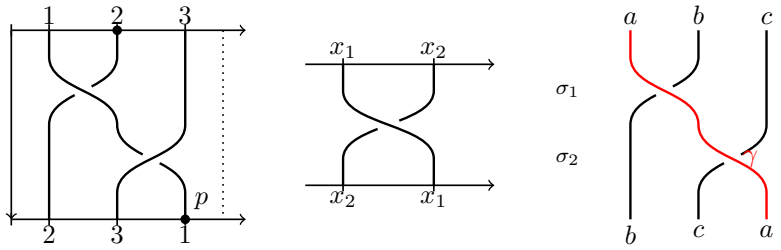


Labels with parentheses or spaces. `x_1`, `V/W`, `X^{\otimes 2}`, `\frac{A}{B}`, `x_{12}` are single atoms. Wrap anything ambiguous: `{V(W)}` or `\atom{f(x, y)}`. (`\frac {A} {B}` with spaces splits; use `\atom`.)

3.6 Axes, marked points, annotations, crossing labels

`\bax{h}` draws two horizontal axes with ticks at the endpoints; `\bax{frame}` adds the vertical arrow and dotted right edge; flags `front`, `noticks`. `\bpt` marks an endpoint; `\bnote` attaches a label to a strand; `\bset{sigma}` labels the crossings in the left margin.

```
\braid{\bin{1 2 3}\bcr{1 -2}\bax{frame}\bpt{in}{2}{}\bpt{out}{3}{p}} \qquad
\braid{\bin{x_1 x_2}\bcr{1}\bax{h,front}} \qquad
\braid{\bin{a b c}\bcr{1 2}\bnote{1}{\gamma}\bcol{1}{red}\bset{sigma}}
```

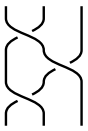


Axes and points are *markers* tied to endpoints; they do not by themselves define a little-discs configuration (Section 6).

3.7 Inline use

Diagrams are display-sized by default. For text use the `inline` preset (a compact band height, spacing, line width and font) and usually `labels=none`; the diagram's centre sits on the math axis.

```
$\sigma_1 = \braid{\bin{a b}\bcr{1}\bset{inline,labels=none}}$, \quad
$\mathrm{id}_2 = \braid{\bn{2}\bset{inline}}$, \quad
$\sigma_1\sigma_2\sigma_1 = \braid{\bin{a b c}\bcr{1 2 1}\bset{inline,labels=none}}$
```

$\sigma_1 =$  , $\mathrm{id}_2 =$  , $\sigma_1\sigma_2\sigma_1 =$ 

4 Composition: an inner braid replaces a strand

`\bsub{k}{...}` replaces strand k of the outer braid by an inner braid. Mathematically the strand is *cabled*: it becomes m parallel strands (a *ribbon*) that cross other strands as one block, and the inner braid is inserted into the ribbon. The boundary configurations of the result are those of the outer braid with the leaf for strand k replaced by the inner configuration (so $x\,y\,z$ with $y \leftarrow (a\,b)$ becomes $x\,(a\,b)\,z$). Strand identities of the result: $1..k{-}1$ unchanged, the ribbon $k..k{+}m{-}1$, later strands shifted by $m{-}1$. The inner braid accepts `\bin`, `\bout`, `\bn`, `\bcr`, `\bcol`; it is validated like any braid. Several `\bsub` may be used in one braid (they refer to outer strand numbers).

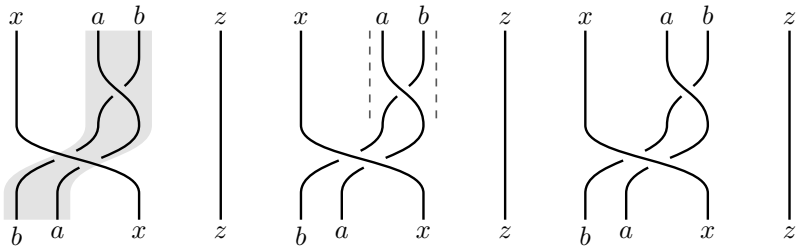
Two **explicit modes** mark the replaced strand; both are chosen with `\bset{compose=...}` or globally with `\braidsetup`:

`compose=shadow` **(Mode 1, default)** a light background behind the ribbon along its *whole* path; shows precisely which part of the outer braid the inner braid replaces. Prefer it when the ribbon crosses other strands, or when you want the replaced strand to stay visible after the inner braid.

`compose=separator` **(Mode 2)** the inner braid replaces the strand and two dashed lines on either side (by default only alongside the inner braid) set it apart from its neighbours. Prefer it for the "box inside a strand" reading, or when shading would clutter dense diagrams.

`compose=none` no marking.

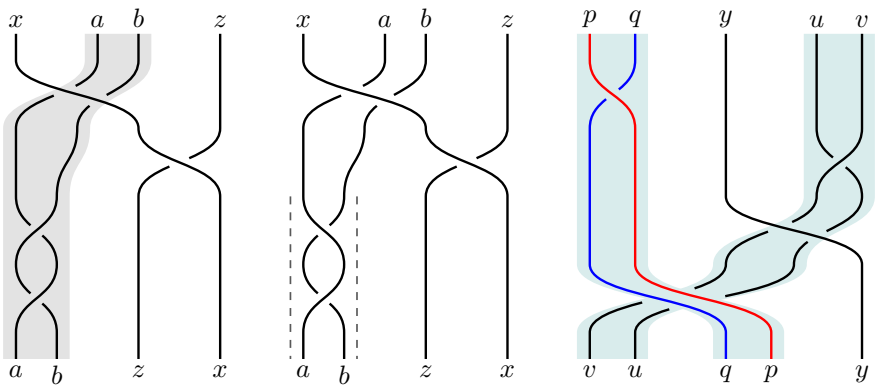
```
\braid{\bin{x y z}\bcr{1}\bsub{2}{\bin{a b}\bcr{1}}\bset{compose=shadow}} \qquad
\braid{\bin{x y z}\bcr{1}\bsub{2}{\bin{a b}\bcr{1}}\bset{compose=separator}} \qquad
\braid{\bin{x y z}\bcr{1}\bsub{2}{\bin{a b}\bcr{1}}\bset{compose=none}}
```



4.1 Where the inner braid sits, how far the marking runs, appearance

`at=top|bottom|`j	the inner braid is inserted at the top (default), at the bottom, or after j outer layers; the boundaries do not depend on it
`extent=auto|full|inner`	how far shadow/separators run: `auto` = full for shadow, inner part for separators (full if the inner braid is trivial)
`shadowcolor`	shadow colour (default `gray!22`)
`sepstyle`, `sepcolor`	dash style (default `dashed`), colour
`regionpad`	distance of shadow edge / separators from the outer strands (default 0.17cm; same for both modes)

```
\braid{\bin{x y z}\bcr{1 2}\bsub{2}{\bin{a b}\bcr{-1 -1}}\bset{at=bottom}} \qquad
\braid{\bin{x y z}\bcr{1 2}\bsub{2}{\bin{a b}\bcr{1 -1}}\bset{compose=separator,at=
    bottom}} \qquad
\braid{\bin{x y z}\bcr{2 1}\bsub{1}{\bin{p q}\bcr{1}\bcol{1}{red}\bcol{2}{blue}}\bsub
    {3}{\bin{u v}\bcr{-1}}%
      \bset{shadowcolor=teal!15}}
```



In the last diagram two strands are replaced; the inner colours override the colour of the outer strand. Composition is also available in Python: `outer.compose_at(k, inner, at="top")`.

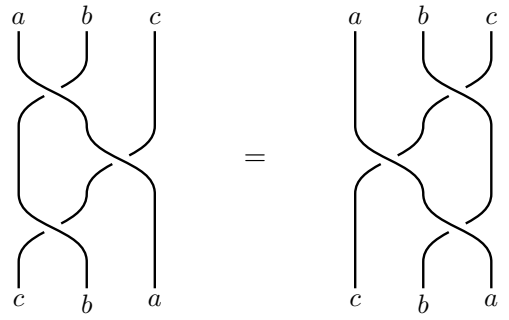
5 Standard identities

`\braidid{name}{options}` draws standard diagrams without any coordinates. Available names: `ybe`, `far`, `inverse`, `crossings`, `assoc`, `pentagon`, `hexagon`.

5.1 Braid relations and basic crossing identities

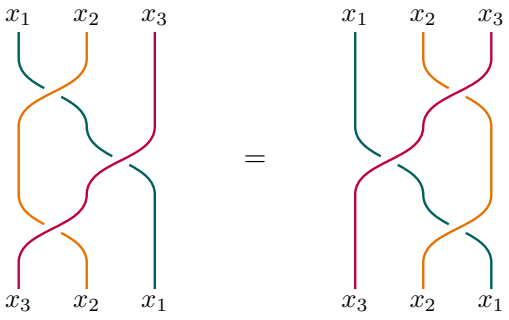
Options: `labels` (space separated), `colors` (one per strand), `sign` (`+`/`-`), `rel` (TeX shown between pictures), `gap`, and any style option (`width`, `lw`, `discs`, ...).

```
\braidid{ybe}{}
```



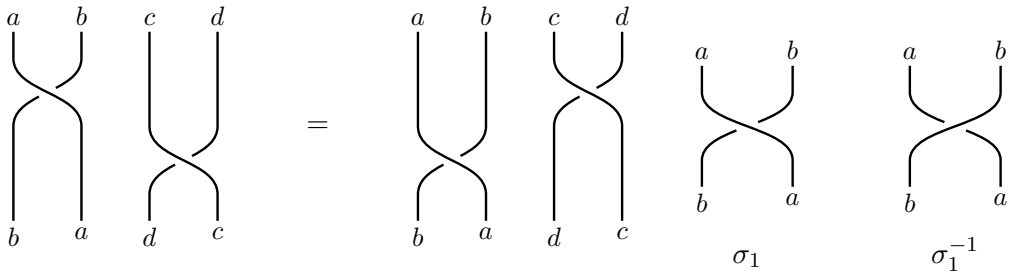
Yang–Baxter / braid relation $\sigma_1\sigma_2\sigma_1 = \sigma_2\sigma_1\sigma_2$.

```
\braidid{ybe}{labels={x_1 x_2 x_3},colors={teal!75!black orange!90!black purple},sign
    =-}
```



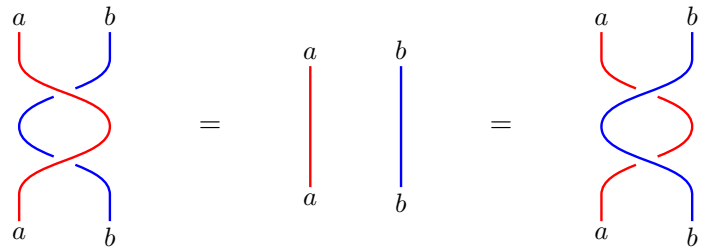
The same relation with inverse generators, own labels and strand colours.

```
\braidid{far}{} \qquad \braidid{crossings}{}
```



Far commutativity $\sigma_1\sigma_3 = \sigma_3\sigma_1$ and the two basic crossings.

```
\braidid{inverse}{colors={red blue}}
```

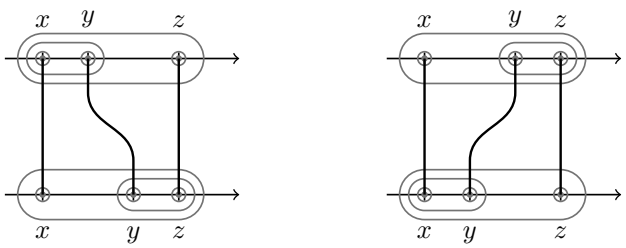


$\sigma\sigma^{-1} = \mathrm{id} = \sigma^{-1}\sigma$. The same strand is over at both crossings (a Reidemeister II move), which is why the picture really is the identity.

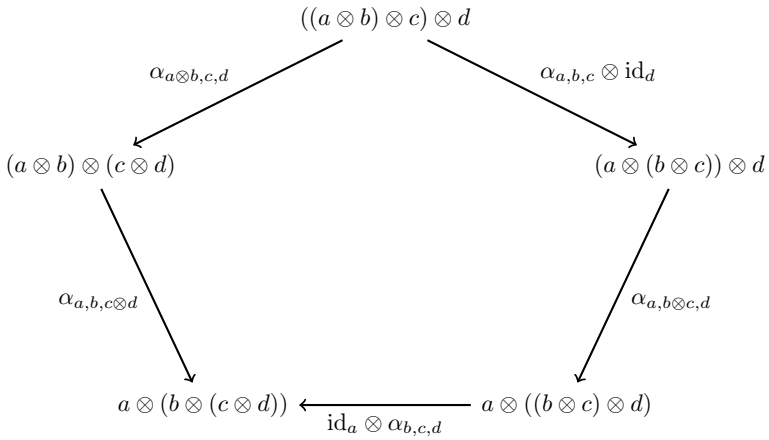
5.2 Associativity, pentagon, hexagon

`assoc` draws the associator $(ab)c \to a(bc)$ and its inverse as braids between differently parenthesized configurations (with discs). `pentagon` and `hexagon` are commutative diagrams whose vertices are all parenthesizations; nodes, arrows and labels are generated and the polygon is sized to fit the labels. Options: `objects`, `op` (tensor sign, `op={}` for juxtaposition), `assoc` (associator symbol), `braiding` (braiding symbol), `radius`, `aspect`, `nodestyle` and `edgestyle` (TikZ keys), `color`, `font`, `scale`.

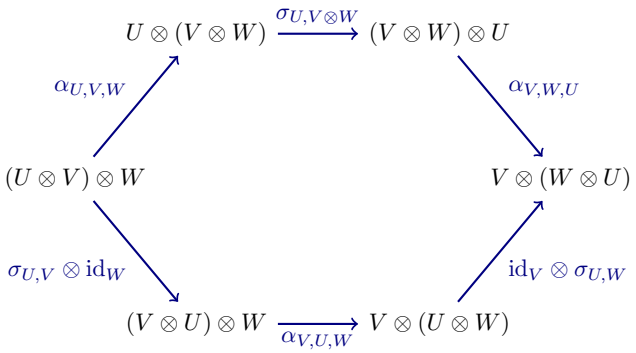
```
\braidid{assoc}{labels={x y z}}
```



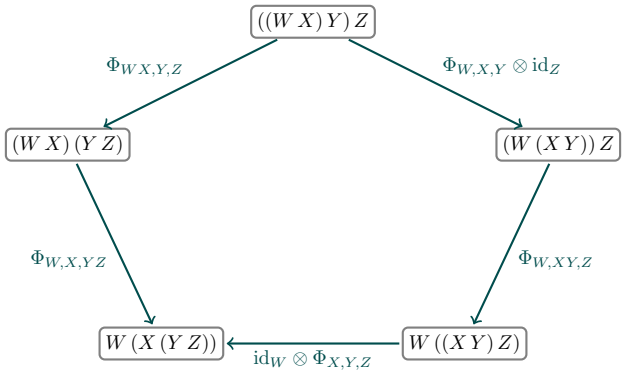

```
\braidid{pentagon}{scale=0.9}
```




```
\braidid{hexagon}{objects={U V W},braiding={\sigma},scale=0.9,color=blue!50!black}
```




```
\braidid{pentagon}{objects={W X Y Z},op={},assoc={\Phi},color=teal!60!black,%
  nodestyle={draw=gray,rounded corners=2pt,inner sep=3pt},scale=0.75}
```



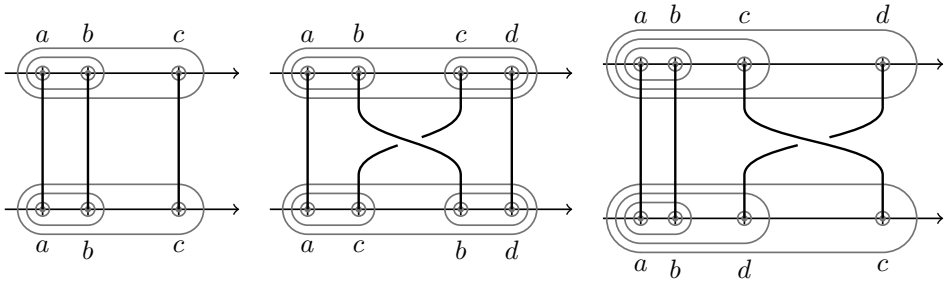
The pentagon commutes in the sense that both paths from $((ab)c)d$ to $a(b(cd))$ pass through trees with the same leaf order; the hexagon is the first hexagon axiom of a braided monoidal category (associators α and braidings c). Wide diagrams: lower `scale` or put them on their own line.

6 Little 2-disc diagrams

The little 2-discs operad is pictured with discs centred on an axis: a configuration $(ab)c$ is two nested levels of discs. `\bset{discs}` (or `discs=in|out|both`) draws, for the input (top) and

output (bottom) configuration, **one disc per node of the parenthesization tree**: leaves are small discs around the endpoints, every parenthesized group is a larger disc that contains exactly the discs of its members, and the whole configuration is the outermost disc. Containment mirrors the parentheses exactly (checked in the test-suite: ancestor $\Leftrightarrow$ contains; unrelated groups are disjoint). Discs imply the horizontal axis; labels are placed outside the outermost disc.

```
\braid{\bin{((a b) c)}\bset{discs}} \quad \braid{\bin{(a b)(c d)}\bcr{2}\bset{discs}} \
    quad
\braid{\bin{((a b) c) d}\bcr{3}\bset{discs,width=4.2cm}}
```

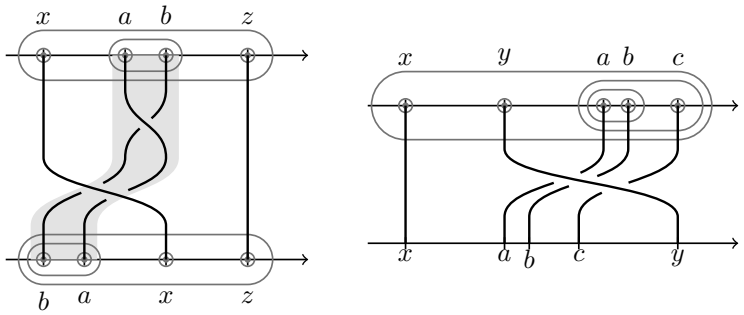


In $((ab)c)$: a and b lie in an inner disc; that disc together with c lies in the outer disc.

6.1 Operadic composition

Combining `\bsub` with discs shows $\gamma(\sigma;\tau)$: the disc of the inserted configuration sits inside the disc slot of the replaced strand, at the top and (for the output configuration) at the bottom.

```
\braid{\bin{x y z}\bcr{1}\bsub{2}{\bin{a b}\bcr{1}}\bset{discs,compose=shadow}} \qquad
\braid{\bin{x y z}\bcr{2}\bsub{3}{\bin{(a b) c}}\bset{discs=in,compose=none}}
```



The marks of Section 4 and the discs can be combined or used separately. **Limitation:** BraidTeX draws the *pictures*; it does not model configuration spaces, operadic composition maps or their relations.

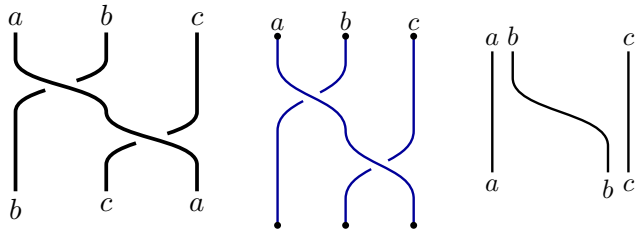
7 Customizing

Options go in `\bset{...}` (one braid) or `\braidsetup{...}` (all following braids; it is linear, not scoped by TeX groups). `\bset` overrides `\braidsetup`. Lengths accept `cm mm pt in` (default cm). Changing style never changes the braid or its boundary configurations.

`width`, `height`	total size (defaults: from strand count and word length)
`spacing`, `minspan`	default endpoint distance (0.9cm) and minimum span (1.2cm)
`band`, `pad`, `margin`	height per crossing layer (0.9cm), straight stub top/bottom, horizontal margin
`lw`	strand line width (0.9pt)
`gap`	clearance around over-strands (0.07cm)
`labels`	`both` (default), `in`, `out`, `none`
`fontsize`	`tiny` … `large`
`color`, `palette`	main strand colour; default per-strand scheme
`dots`, `sigma`	endpoint markers; crossing labels $\sigma_i^{\pm 1}$ (left margin)
`rho`, `wmin`	nesting ratio and smallest relative gap of the hierarchical layout
`inpos`, `outpos`	custom normalized endpoint positions, e.g. `inpos={0 .2 1}`
`compose`, `at`, `extent`, `shadowcolor`, `sepstyle`, `sepcolor`, `regionpad`	composition modes (Section 4)
`discs`	little-disc nesting (Section 6)
`inline` / `size=inline`	compact preset for use in running text

7.1 Examples

```
\braid{\bin{a b c}\bcr{1 2}\bset{width=3.4cm,lw=1.4pt,band=0.7cm,fontsize=normalsize}}
    \qquad
\braid{\bin{a b c}\bcr{1 2}\bset{dots,labels=in,color=blue!60!black}} \qquad
\braid{\bin{a b c}\bset{inpos={0 0.15 1},outpos={0 0.85 1}}\bcr{}}
```



Document defaults. For a whole thesis put in the preamble, e.g. `\braidsetup{lw=0.8pt, compose=separator, palette}`; override locally with `\bset`.

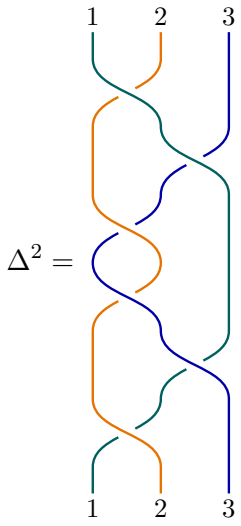
Geometry in one paragraph. Each strand is a chain of cubic Bézier curves with vertical tangents at the layer boundaries (smooth, no kinks; minimum radius of curvature about 1.8 mm at the default spacing). All strands in a layer share the same parametrization, so two strands that do not swap never meet and two that swap cross *exactly once*: a diagram always has exactly as many crossings as generators, whatever the input/output spacings. The under-strand is cut by a true vector gap, not masked in white.

8 Gallery

Each example is complete: paste it into a document that loads `braidtex` and run `braidtex compile`.

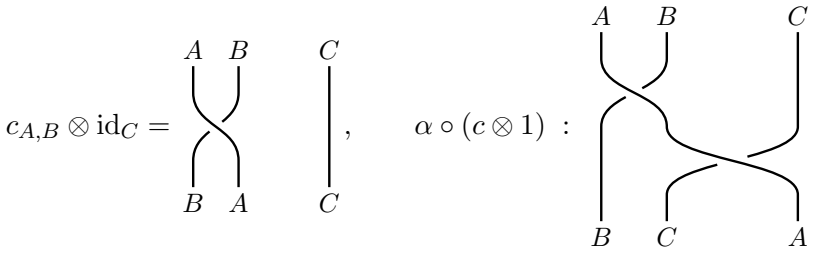
Full twist

```
\Delta^2 = \braid{\bin{1 2 3}\bcr{1 2 1 1 2 1}\bset{palette}}
```



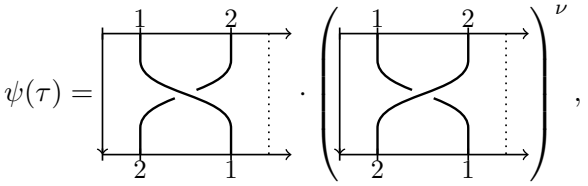
Braided hexagon pieces and a morphism in the parenthesized braid category

```
c_{A,B}\otimes \mathrm{id}_C = \braid{\bin{(A B) C}\bcr{1}}, \qquad
\alpha\circ(c\otimes 1) \;:\; \braid{\bin{(A B) C}\bcr{1 2}\bout{(B C) A}\bset{width
    =3.6cm}}
```



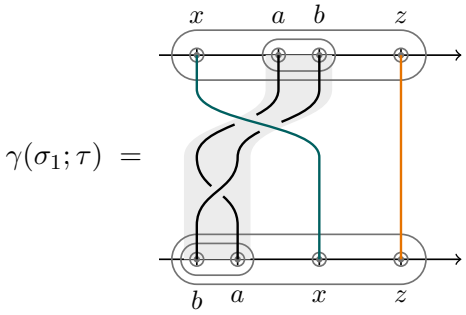
Equation with axes (as in the thesis picture)

```
\psi(\tau) = \braid{\bin{1 2}\bcr{1}\bax{frame}} \cdot
\left(\braid{\bin{1 2}\bcr{-1}\bax{frame}}\right)^{\nu},
```



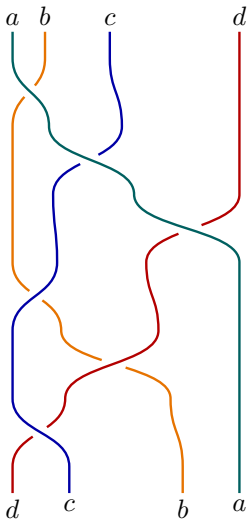
Operadic composition with discs

```
\gamma(\sigma_1;\tau)\;=\;
\braid{\bin{x y z}\bcr{1}\bsub{2}{\bin{a b}\bcr{-1}}\bcol{1}{teal!75!black}\bcol{3}{
    orange!90!black}%
      \bset{discs,at=bottom,shadowcolor=gray!15}}
```



Many crossings, different input and output trees

```
\braid{\bin{((a b) c) d}\bcr{1 2 3 -1 -2 1}\bout{(d c)(b a)}\bset{width=4cm,palette}}
```



9 Troubleshooting

A grey box "unprocessed braid" appears. You ran `pdflatex` directly. Use `braidtex compile main.tex`.

"Missing braid asset". The transformed source refers to `braids/bt-...`, but LaTeX runs elsewhere. Run from the project directory, or just use `braidtex compile`. After moving the project run it again (assets are regenerated).

"output endpoint j is labeled ...". `\bout` contradicts the strand permutation. The message lists the expected output labels. For a purely visual change use `\blab{out}{...}`.

"generator $\#k = g$ is invalid". With n strands only $1 \le |i| \le n-1$ exist. If you gave no `\bin`/`\bn`, n is inferred as $\max|i|+1$.

A label with parentheses is split into groups. Wrap it: `{f(x)}` or `\atom{f(x), y}`.

Diagram too large inline. `\bset{inline,labels=none}`, or set `height`/`band`/`width`.

Warnings about endpoints closer than 2.5 mm or a gap reaching the end of a layer. Increase `width`/`band`, lower the nesting weight (`wmin`), or reduce `gap`.

"regionpad is too large". The shadow/separators would touch a neighbouring strand: reduce `regionpad` or widen the diagram.

Dashed separators look cluttered. `extent=full` draws them along the whole ribbon, across other strands; prefer the default, or use `compose=shadow` for long ribbons.

Pentagon/hexagon too wide. `scale=0.8`, or put it on its own line.

Changes in `\braidsetup` seem ignored. It applies to braids *after* it in the source and is not group-scoped.

`\braid` inside `\newcommand` or another macro argument. Not supported: the scanner reads source text, not expanded macros. Put `\braid` directly in the document, or generate the macro text before running `braidtex`.

Stale pictures. `braidtex compile -f` regenerates everything; `braidtex clean` deletes assets.

Conflict with another package. The CTAN package `braids` also defines `\braid`; do not load both.

Does BraidTeX need shell escape? No. It is never used; the engine is run with `-no-shell-escape`.

10 Reference

10.1 Command line

```
braidtex render --expr '\bin{a b}\bcr{1}' [--format tikz|standalone|svg|pdf] [-o FILE]
braidtex render --json spec.json --format pdf -o fig.pdf # standalone figure
braidtex render --identity pentagon --opts 'objects={a b c d}' -o p.tex
braidtex expr '<latex fragment with \braid{..}>' # transform a fragment
braidtex transform main.tex [--check] [--stdout] [--build DIR] # preprocess only
braidtex compile main.tex [--engine pdflatex|lualatex|xelatex] [-f] [-v]
braidtex clean [DIR] braidtex sty braidtex --version
```

Common options: `-assets DIR` (default `braids/`), `-f` force, `-v` verbose. Errors in the source are reported as `file:line:col: message` with exit status 2.

10.2 Python

```
from braidtex import Braid, Style, Cache, render_tikz
outer = Braid([1], "x y z")
inner = Braid([1], "a b")
c = outer.compose_at(2, inner, at="top") # boundaries: x (a b) z -> (b a) x z
c.permutation, c.regions # strand k ends at output slot p[k-1]
open("fig.tikz", "w").write(render_tikz(c.replace(style=Style(compose="separator",
    discs="both"))))
Cache("braids").ensure(c, "pdf") # cached standalone PDF
```

10.3 Limitations

`\braid` needs the preprocessing step; only `\input`/`\include` are followed; named/mixed xcolor colours only; SVG output shows plain-text labels and omits composition marks and discs; pentagon and hexagon are the standard commutative diagrams (first hexagon axiom) with text labels; BraidTeX draws little-disc *pictures* but does not model configuration spaces or operad composition.